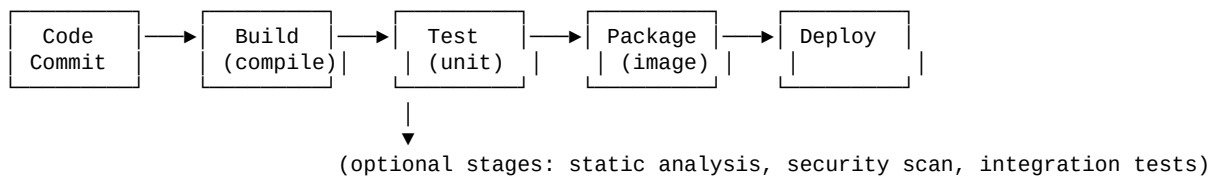


CI/CD PIPELINE & GITHUB ACTIONS

1. Introduction to CI/CD Pipeline

CI/CD (Continuous Integration / Continuous Delivery or Deployment) is a software development practice that automates building, testing, and delivering code changes. The pipeline starts with a code commit and ends with deployment to production.

Basic CI/CD Pipeline Flow:

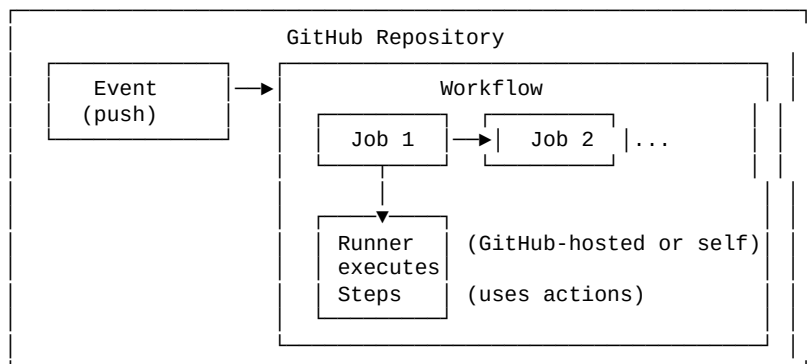


- **Continuous Integration (CI):** Developers merge code frequently into a shared branch; automated builds and tests run to catch issues early.
- **Continuous Delivery (CD):** After CI, the code is automatically packaged and deployed to a staging environment; manual approval may be required for production.
- **Continuous Deployment:** Every change that passes automated tests is deployed to production automatically (no manual gate).

2. Introduction to GitHub Actions

GitHub Actions is a CI/CD platform built into GitHub. It allows you to automate workflows that build, test, and deploy your code right from your repository. Workflows are triggered by GitHub events (push, pull_request) and can run on various runners.

GitHub Actions High-Level:



3. GitHub Actions Components Overview

- **Workflow:** A configurable automated process defined in YAML file (located in `.github/workflows/`).
- **Event:** A specific activity that triggers a workflow (e.g., push, pull_request, schedule).
- **Job:** A set of steps that execute on the same runner. Jobs can run sequentially or in parallel.
- **Step:** An individual task (either a shell command or an action).
- **Action:** A reusable, custom unit of code (e.g., `actions/checkout@v4`).

- **Runner:** A server that runs the workflow jobs (GitHub-hosted or self-hosted).

4. Workflow

A workflow is a YAML file that defines the CI/CD pipeline. It resides in the ``.github/workflows/`` directory. Multiple workflows can exist in a repository.

```
# .github/workflows/ci.yml
name: CI Pipeline
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run a script
        run: echo "Hello, world!"
```

- **name:** Display name of the workflow (optional).
- **on:** Defines the event(s) that trigger the workflow.
- **jobs:** Contains one or more jobs.
- **env:** Environment variables for the whole workflow.
- **defaults:** Default settings for all jobs (e.g., shell).
- **concurrency:** Ensures only one workflow run at a time for a given group.

5. Events

Events are triggers that start a workflow. Common events:

Event	Description
push	When code is pushed to a branch or tag.
pull_request	When a PR is opened, synchronized, or closed.
schedule	Runs at specific times using cron syntax (UTC).
workflow_dispatch	Manually triggered workflow from GitHub UI.
release	When a GitHub Release is published.
issues / issue_comment	Issue-related events.
repository_dispatch	Triggered via GitHub API.

```
# Example with multiple events and branch filters
on:
  push:
    branches: [ main, develop ]
    tags: [ 'v*' ]
  pull_request:
    branches: [ main ]
    types: [ opened, synchronize, reopened ]
  schedule:
    - cron: '0 2 * * *' # daily at 2am UTC
  workflow_dispatch: # manual trigger
  inputs:
    environment:
      description: 'Deploy env'
      default: 'staging'
```

6. Jobs

A job is a group of steps that execute on the same runner. Jobs can depend on other jobs (`needs`), run conditionally, and have their own environment variables.

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      build-id: ${ steps.set-id.outputs.id }
    steps:
      - name: Generate ID
        id: set-id
        run: echo "id=$(date +%s)" >> $GITHUB_OUTPUT
  test:
    needs: build
    runs-on: ubuntu-latest
    if: success()
    steps:
      - run: echo "Testing after build"
  deploy:
    needs: [build, test]
    runs-on: self-hosted
    environment: production
    steps:
      - run: ./deploy.sh
```

- **runs-on:** Specifies the runner type (e.g., `ubuntu-latest`, `windows-latest`, `self-hosted`).
- **needs:** Defines job dependencies (order and waiting).
- **if:** Conditional execution (e.g., `if: github.ref == 'refs/heads/main'`).
- **environment:** Deployment environment (e.g., `staging`, `production`) with optional approval rules.
- **outputs:** Map of outputs that can be consumed by dependent jobs.
- **strategy.matrix:** Run job multiple times with different variables (parallel testing).

7. Actions

Actions are reusable units of code that can be composed into steps. They can be from GitHub Marketplace, from your own repository, or from public repositories.

```

steps:
  # Public action from GitHub
  - uses: actions/checkout@v4

  # Action from a specific repository
  - uses: docker/setup-buildx-action@v3

  # Custom action in ./my-action directory
  - uses: ./my-action

  # Shell command (not an action, but a step)
  - name: Run custom command
    run: |
      echo "Hello"
      npm install

  # Using environment variables and secrets
  - name: Deploy to AWS
    uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_KEY }
      aws-secret-access-key: ${ secrets.AWS_SECRET }
      aws-region: us-east-1

```

Creating custom actions: You can create a Docker container action, JavaScript action, or composite action.

8. Runners

A runner is a server that runs the jobs. GitHub Actions supports two types:

8.1 GitHub-Hosted Runners

Managed by GitHub, automatically provisioned for each job. Available operating systems: Ubuntu Linux, Windows, macOS.

```

runs-on: ubuntu-latest      # Ubuntu 22.04
runs-on: windows-2022      # Windows Server 2022
runs-on: macos-13          # macOS Ventura

```

- **Advantages:** No maintenance, automatically updated tools, isolated per job, free for public repositories (some free minutes for private).
- **Limitations:** Standardised image (cannot install custom packages at VM level, only per job), less control over hardware.
- **Specs (example ubuntu-latest):** 2-core CPU, 7 GB RAM, 14 GB SSD.

8.2 Self-Hosted Runners

Runners you host on your infrastructure (VM, on-prem, cloud). Register with GitHub and run the runner application.

```
# Register a self-hosted runner
# Download runner from GitHub UI (Settings → Actions → Runners)
./config.sh --url https://github.com/owner/repo --token --labels custom-label
./run.sh

# Use in workflow
runs-on: [self-hosted, custom-label]
```

- **Advantages:** Full control over hardware, OS, software, caching; can access private network resources; no per-minute billing.
- **Disadvantages:** You manage updates, security, scaling, and availability.
- **Use cases:** Large builds, GPU workloads, accessing private VPC, specific legacy tools.
- **Security warning:** Self-hosted runners can access your infrastructure; avoid running untrusted code.

8.3 Runner Type Comparison

Feature	GitHub-Hosted Runner	Self-Hosted Runner
Setup & maintenance	Zero (fully managed)	Manual (installation, updates, scaling)
Cost	Free for public repos; paid minutes for private	No additional GitHub cost (but pay for your infrastructure)
Custom software	Limited to preinstalled tools; install per job	Full control, pre-install any software
Hardware	Fixed standard specs	Custom (more RAM, CPU, GPU, large storage)
Network access	Public internet only (cannot access private networks)	Can access on-prem or private cloud resources
Job concurrency	One job per runner instance (auto-scaling)	One concurrent job per runner process

9. Complete Example Workflow

```
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
  workflow_dispatch:

env:
  DOCKER_REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - name: Install dependencies
        run: npm ci
      - name: Run tests
        run: npm test
      - name: Upload test report
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: test-results.xml

  docker-build-and-push:
    needs: build-and-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ${ env.DOCKER_REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${ env.DOCKER_REGISTRY }/${ env.IMAGE_NAME }:latest

  deploy:
    needs: docker-build-and-push
    runs-on: self-hosted
    environment: production
    steps:
      - name: Deploy to production
        run: ./deploy-script.sh
```

Key Takeaways:

- CI/CD pipelines automate the development lifecycle from commit to deployment.
 - GitHub Actions provides integrated CI/CD using YAML workflows.
 - A workflow consists of events, jobs, and steps; jobs can run in parallel or sequentially.
 - Actions are reusable building blocks (official, community, or custom).
 - Runners execute jobs: GitHub-hosted are convenient, self-hosted give full control.
-